Sumario

¿Qué es YAML?....4
1.1 Origen y significado de YAML Ain"t Markup Language....4
1.2 Diferencias con XML y JSON....4
YAML vs JSON....4
YAML vs XML....5
1.3 Usos comunes de YAML....6
Archivos de configuración....6
Infraestructura como código (DevOps)....6
Definición de pipelines de CI/CD....6
Intercambio de datos estructurados....6
Instalación y herramientas recomendadas....6
2.1 Editores y plugins....6
Visual Studio Code (VSCode)....6
Sublime Text....7
IntelliJ IDEA / PyCharm....7
Otros editores útiles....7
2.2 Validadores en línea....7
2.3 Recomendación práctica....7
Reglas básicas de YAML....8
3.1 Indentación y espacios....8
3.2 Sensibilidad a mayúsculas....9
3.3 Comentarios....9
3.4 Buenas prácticas....10
Tipos de datos primitivos en YAML....10
4.1 Strings (cadenas de texto)....10
a) Strings simples sin comillas....10
b) Strings con comillas dobles "..."....10
c) Strings con comillas simples '...'....10
d) Strings multilínea....10
4.2 Números....11
4.3 Booleanos....11
4.4 Null (valor nulo)....11
4.5 Buenas prácticas....11
Estructuras básicas en YAML....12
5.1 Listas (arrays)....12
5.2 Diccionarios (maps)....13
5.3 Combinación de listas y diccionarios....13
5.4 Buenas prácticas....14
Referencias y anclas (& y *)....15
6.1 Anclas (&)....15
6.2 Referencias o alias (*)....15
6.3 Uso con listas....15
6.4 Sobrescritura de valores....16
6.5 Caso práctico: Docker Compose....16
6.6 Buenas prácticas....17
Patrones de reutilización con YAML....17

7.1 Reutilización de configuraciones base....17
7.2 Reutilización en listas....18
7.3 Reutilización de metadatos....18
7.4 Reutilización en Docker Compose....18
7.5 Patrón de valores por defecto....19
7.6 Buenas prácticas al reutilizar....19
7.7 Ventajas de estos patrones....19
Documentos múltiples (---)....20
8.1 ¿Cuándo usar documentos múltiples?....20
8.2 Ejemplo simple....20
8.3 Ejemplo en Kubernetes....20
8.4 Separador final (...)....22
8.5 Buenas prácticas....22
8.6 Ventajas....22
8.7 Limitaciones....22
Tags de tipo (!!str, !!int, etc.)....23
9.1 ¿Qué es un tag de tipo?....23
9.2 Principales tags de tipo....23
a) !!str — cadenas de texto....23
b) !!int — enteros....23
c) !!float — decimales....23
d) !!bool — booleanos....23
e) !!null — valores nulos....23
f) !!seq — listas (secuencias)....24
g) !!map — diccionarios (mapas)....24
9.3 Casos prácticos....24
9.4 Buenas prácticas....24
9.5 Ventajas y limitaciones....24
Bloques literales y doblados....24
10.1 Bloque literal (|)....25
10.2 Bloque doblado (>)....25
10.3 Comparación rápida....25
10.4 Control de saltos finales (|+, |-, >+, >-)....26
10.5 Buenas prácticas....26
10.6 Casos de uso....26
Archivos de configuración de aplicaciones....27
11.1 Docker Compose (docker-compose.yml)....27
11.2 Kubernetes (deployment.yaml y service.yaml)....28
Deployment (deployment.yaml)....28
Service (service.yaml)....29
11.3 Deployment y Service juntos....29
11.4 Docker Compose vs Kubernetes....30
11.5 Buenas prácticas....30
Uso en frameworks y lenguajes....30
12.1 Spring Boot....31
12.2 Python con PyYAML....31
12.3 Node.js con js-yaml....32
12.4 Comparación rápida....33

12.5 Buenas prácticas....33
Validación y linting de YAML....33
13.1 Herramientas automáticas....33
a) YAML Lint (yamllint CLI)....33
b) Extensión YAML de Red Hat (VSCode)....34
c) Validación con JSON Schema....34
d) Validación en pipelines CI/CD....34
13.2 Errores comunes....34
Uso de tabulaciones....34
Indentación inconsistente....35
Valores ambiguos sin comillas....35
Separadores de documentos mal usados....35
Listas mal indentadas....35
13.3 Buenas prácticas....35
Conversión entre YAML y JSON....36
14.1 Ejemplo de equivalencia....36
14.2 Ventajas al convertir....36
14.3 Herramientas online....37
14.4 Conversión en Python (PyYAML)....37
14.5 Conversión en Node.js (js-yaml)....37
14.6 Buenas prácticas....38
Estilo y convenciones de nombres en proyectos reales....38
15.1 Convenciones para las claves....39
15.2 Indentación consistente....39
15.3 Listas alineadas....39
15.4 Uso de comillas....39
15.5 Orden lógico de las claves....39
15.6 Convenciones en proyectos DevOps....40
15.7 Comentarios útiles....40
15.8 Integración con linters....40
15.9 Beneficios....41
Seguridad y limitaciones de YAML....41
16.1 Riesgos de seguridad....41
a) Ejecución de código malicioso....41
b) Inyección de datos....41
c) Exposición de credenciales....41
16.2 Limitaciones técnicas....42
a) Sensibilidad a la indentación....42
b) Ambigüedad en valores....42
c) Archivos grandes y confusos....42
d) Comentarios limitados....42
e) Rendimiento....42
16.3 Buenas prácticas de seguridad....42
16.4 Buenas prácticas ante limitaciones....42

¿Qué es YAML?

1.1 Origen y significado de YAML Ain''t Markup Language

YAML es un formato de serialización de datos creado en el año 2001 por Clark Evans, Ingy döt Net y Oren Ben-Kiki.
Su nombre proviene originalmente de Yet Another Markup Language (Otro lenguaje de marcado más). Sin embargo, poco tiempo después los propios creadores aclararon que en realidad YAML no debe entenderse como un lenguaje de marcado, sino como un lenguaje de serialización de datos, y modificaron el acrónimo a:
👉 YAML Ain''t Markup Language (YAML no es un lenguaje de marcado).
Con esto quisieron remarcar que la intención principal no es dar formato al texto (como XML) sino representar datos estructurados de una forma sencilla, legible para humanos y fácil de procesar por las computadoras.

1.2 Diferencias con XML y JSON

YAML se compara habitualmente con XML y JSON, que son otros formatos muy usados para representar datos. Veamos las principales diferencias:

YAML vs JSON

•Legibilidad: YAML está pensado para que sea más fácil de leer y escribir por personas. No requiere llaves {}, corchetes [] ni comillas en todos los casos.
•Simplicidad: la estructura se define por indentación (espacios), lo cual hace que el archivo sea más limpio.
•Riesgo de errores: la indentación incorrecta genera errores difíciles de detectar.
•Compatibilidad: YAML es un superconjunto de JSON, lo que significa que cualquier documento JSON válido también es YAML válido.
Ejemplo de una misma lista de productos en JSON y YAML:
JSON:

```
{
  "productos": [
    { "codigo": 1, "descripcion": "Teclado", "precio": 15000 },
    { "codigo": 2, "descripcion": "Mouse", "precio": 9000 }
  ]
}
```

YAML:

```
productos:
```

```yaml
  - codigo: 1
    descripcion: "Teclado"
    precio: 15000
  - codigo: 2
    descripcion: "Mouse"
    precio: 9000
```

YAML vs XML

•Menos verboso: YAML evita las etiquetas de apertura y cierre que hacen que XML sea más extenso y difícil de leer.
•Más cercano a la escritura natural: YAML usa indentación y listas con guiones (-), similar a cómo escribiríamos notas o esquemas.
•Menos formalidad: XML tiene un estándar muy estricto con esquemas de validación (DTD, XSD), lo que lo hace más robusto en entornos que requieren máxima seguridad y validación rígida.
•Practicidad: YAML es más rápido para configuración y lectura humana; XML sigue siendo más usado en contextos de interoperabilidad empresarial.
Ejemplo en XML vs YAML:
XML:

```xml
<productos>
  <producto>
    <codigo>1</codigo>
    <descripcion>Teclado</descripcion>
    <precio>15000</precio>
  </producto>
  <producto>
    <codigo>2</codigo>
    <descripcion>Mouse</descripcion>
    <precio>9000</precio>
  </producto>
</productos>
```

YAML:

```yaml
productos:
  - codigo: 1
    descripcion: Teclado
    precio: 15000
  - codigo: 2
    descripcion: Mouse
    precio: 9000
```

1.3 Usos comunes de YAML

Hoy en día YAML se ha convertido en un estándar de facto para la definición de configuraciones en múltiples herramientas. Algunos de los usos más importantes son:

Archivos de configuración

En proyectos modernos, la mayoría de frameworks y herramientas usan YAML para definir parámetros de configuración. Ejemplo: application.yml en Spring Boot.

Infraestructura como código (DevOps)

Es muy utilizado en Docker Compose (docker-compose.yml) y Kubernetes (deployment.yaml, service.yaml). Permite definir contenedores, servicios, redes y volúmenes en un solo archivo legible.

Definición de pipelines de CI/CD

GitHub Actions, GitLab CI/CD y Azure DevOps utilizan archivos .yml para definir flujos de integración continua y despliegue automatizado.

Intercambio de datos estructurados

En APIs y microservicios puede emplearse para configurar esquemas, plantillas o datos iniciales. Aunque JSON sigue siendo más popular para transmisión de datos, YAML se prefiere en los archivos de configuración que los acompañan.
En resumen: YAML es un formato de datos pensado para ser simple, legible por humanos y ampliamente adoptado en configuraciones y DevOps.

Instalación y herramientas recomendadas

Aunque YAML es solo texto plano, trabajar con herramientas adecuadas facilita escribir, organizar y validar archivos con indentación impecable.

2.1 Editores y plugins

Hoy en día casi cualquier editor soporta YAML, pero algunos destacan por su ecosistema y validaciones integradas:

Visual Studio Code (VSCode)

Visual Studio Code es uno de los editores más usados entre desarrolladores y equipos DevOps.

Para obtener autocompletado y validaciones en tiempo real conviene instalar la extensión YAML (Red Hat). Esta extensión permite validar documentos contra esquemas JSON Schema, ideal al trabajar con Kubernetes, GitHub Actions o Docker.
Configura la recomendación de la extensión en tu espacio de trabajo:

```
{
  "recommendations": [
    "redhat.vscode-yaml"
  ]
}
```

Sublime Text

Sublime Text dispone de paquetes que añaden resaltado y detección básica de errores de indentación. Instálalos desde Package Control buscando opciones como YAML/Ansible o YAML support.

IntelliJ IDEA / PyCharm

IntelliJ IDEA y PyCharm reconocen automáticamente archivos .yml y .yaml, con validación, autocompletado y soporte para frameworks como Spring Boot y Kubernetes.

Otros editores útiles

•Notepad++: habilita el complemento de YAML para resaltar sintaxis.
•Vim / NeoVim: extensiones como vim-yaml ayudan a detectar indentación incorrecta.

2.2 Validadores en línea

La mayoría de los errores en YAML provienen de espacios o indentación. Estos validadores permiten pegar el archivo y localizar fallos rápidamente:
•YAML Lint: muestra la línea exacta del problema y ofrece reglas de estilo.
•Code Beautify YAML Validator: valida y convierte entre JSON y YAML.
•Online YAML Tools: incluye utilidades para formatear, minificar y convertir archivos grandes.
•JSON ↔ YAML Converter: conversión instantánea para verificar que no se pierdan datos.

2.3 Recomendación práctica

Para el trabajo diario, combina un editor con soporte de esquemas y un validador rápido:

•Uso cotidiano: VSCode junto con la extensión de Red Hat brinda validación en tiempo real.
•Revisiones puntuales: servicios como YAML Lint o Code Beautify permiten un chequeo inmediato.
•Proyectos grandes: integra un linter como yamllint en tus pipelines de CI para detectar inconsistencias antes del despliegue.
Ejemplo de configuración mínima de yamllint en un proyecto:

```
extends: default
rules:
  line-length:
    max: 120
    level: warning
  truthy:
    allowed-values: ['true', 'false']
```

Con estas herramientas y validadores podrás trabajar con YAML de forma segura y confiable, evitando errores de indentación y sintaxis.

Reglas básicas de YAML

YAML destaca por su legibilidad, pero esa ventaja depende de respetar normas estrictas sobre espacios, mayúsculas y comentarios. Dominar estas reglas desde el inicio evita errores difíciles de diagnosticar.

3.1 Indentación y espacios

En YAML la estructura jerárquica depende de la indentación. Cada nivel se define con espacios, nunca con tabulaciones.
•Usa siempre espacios; las tabulaciones no están permitidas.
•Mantén una cantidad constante (2 o 4 espacios) en todo el archivo.
•Cualquier inconsistencia provocará errores de análisis.
Ejemplo correcto (indentación consistente con 2 espacios):

```
producto:
  codigo: 1
  descripcion: Teclado
  precio: 15000
```

Ejemplo incorrecto (usa una tabulación en lugar de espacios):

```
producto:
        descripcion: Teclado   # <- tabulación no permitida
  precio: 15000
```

Los procesadores YAML informarán un error de sintaxis debido a la tabulación.

Ejemplo incorrecto (mezcla 2 y 3 espacios):

```
producto:
   codigo: 1      # 3 espacios
  descripcion: Teclado  # 2 espacios
   precio: 15000
```

La inconsistencia en la cantidad de espacios dificulta la lectura y provoca errores.

3.2 Sensibilidad a mayúsculas

Las claves y los valores de tipo texto son sensibles a mayúsculas y minúsculas. Producto, producto y PRODUCTO se consideran claves diferentes.

Ejemplo con claves distintas por su capitalización:

```
Producto: Teclado
producto: Mouse
PRODUCTO: Monitor
```

Para evitar confusiones, la convención en proyectos de software y DevOps es usar minúsculas y separar palabras con guiones medios (-) o guiones bajos (_).

Ejemplo recomendado:

```
producto:
  codigo: 1
  nombre_producto: Teclado
  precio: 15000
```

3.3 Comentarios

Los comentarios comienzan con #. Todo lo que siga en la misma línea se ignora al procesar el archivo.

Ejemplo de comentarios en líneas separadas y al final de línea:

```
# Este es un comentario de una sola línea
producto:
  codigo: 1       # Código interno del producto
  descripcion: PC  # Descripción del producto
  precio: 200000   # Precio en pesos
```

No existen comentarios multilínea tipo /* ... */. Para desactivar temporalmente un bloque, comenta línea por línea.

Ejemplo de bloque comentado:

```
#producto:
#  codigo: 2
#  descripcion: Mouse
#  precio: 9000
```

3.4 Buenas prácticas

•Usa siempre espacios en lugar de tabs; dos espacios por nivel es una elección popular.
•Mantén la indentación uniforme en todo el archivo.
•Prefiere minúsculas para las claves y separadores consistentes (- o _).
•Añade comentarios claros y concisos sin saturar el archivo.
Siguiendo estas reglas básicas podrás escribir archivos YAML confiables, fáciles de leer y sin errores de sintaxis.

Tipos de datos primitivos en YAML

En YAML, los tipos de datos primitivos permiten representar valores básicos: cadenas de texto, números, booleanos y valores nulos. Sobre ellos se construyen estructuras más complejas como listas y diccionarios.

4.1 Strings (cadenas de texto)

Los strings son muy frecuentes y pueden escribirse de distintas maneras según el contenido:

a) Strings simples sin comillas

```
nombre: Teclado
marca: Logitech
```

b) Strings con comillas dobles "..."

Útiles para incluir caracteres especiales (como : o #) o secuencias de escape (\n para saltos de línea).

```
descripcion: "Teclado mecánico con retroiluminación RGB"
nota: "Incluye garantía de 2 años\nFabricado en China"
```

c) Strings con comillas simples '...'

Interpretan el texto de forma literal, sin procesar secuencias de escape.

```
mensaje: 'La ruta del archivo es C:\usuario\documentos'
```

d) Strings multilínea

Para bloques de texto largos se usan los estilos literal (|) o doblado (>):
Bloque literal (conserva saltos de línea):

```
descripcion: |
  Este es un teclado mecánico.
  Tiene retroiluminación RGB.
```

```
  Ideal para gaming.
```

Bloque doblado (une líneas con espacios):

```
descripcion: >
  Este es un teclado mecánico.
  Tiene retroiluminación RGB.
  Ideal para gaming.
```

4.2 Números

Los números se escriben sin comillas. YAML soporta enteros, decimales y notación científica.

a) Enteros:

```
cantidad: 50
anio: 2025
```

b) Decimales:

```
precio: 1999.99
descuento: 0.15
```

c) Notación científica:

```
gravedad: 9.81e0
planck: 6.626e-34
```

Importante: si necesitas que un número se trate como texto (por ejemplo, un código postal con ceros a la izquierda), usa comillas: `codigo_postal: "05000"` .

4.3 Booleanos

YAML acepta varias formas para `true` y `false`: true/false, yes/no y on/off. La convención más usada en proyectos de software es true y false en minúsculas.

```
disponible: true
en_stock: yes
activo: on
agotado: false
```

4.4 Null (valor nulo)

Representa ausencia de dato. Se puede escribir como `null`, `Null`, `~` o dejando el valor vacío.

```
cliente: null
direccion: ~
telefono:
```

4.5 Buenas prácticas

•Usa comillas dobles si el texto contiene caracteres especiales.

•Para texto multilínea: `|` mantiene saltos; `>` los une.

•Los números se escriben sin comillas salvo que necesites preservar el formato.
•Prefiere true/false en minúsculas para booleanos.
•Para valores nulos, null es la forma más clara.
Con estos tipos de datos primitivos puedes modelar información básica en YAML de forma clara y segura.

Estructuras básicas en YAML

En YAML los datos se organizan principalmente mediante listas y diccionarios (también llamados mapas u objetos). Estas estructuras permiten modelar información compleja de forma legible.

5.1 Listas (arrays)

Una lista es una colección ordenada en la que cada elemento comienza con un guion (-) precedido por un espacio.
Lista simple de frutas:

```yaml
frutas:
  - Manzana
  - Banana
  - Naranja
```

Equivalente en **JSON**:

```json
{
  "frutas": ["Manzana", "Banana", "Naranja"]
}
```

Lista de números:

```yaml
numeros:
  - 10
  - 20
  - 30
```

Lista multilínea:

```yaml
lenguajes:
  - Python
  - Java
  - JavaScript
  - Go
```

Tip: toda línea de lista usa un guion seguido de un espacio. La indentación debe ser uniforme para evitar errores.

5.2 Diccionarios (maps)

Un diccionario agrupa pares clave-valor. La clave se escribe seguida de dos puntos (:) y el valor a continuación, respetando la indentación.

Diccionario simple:

```
producto:
  codigo: 1
  descripcion: Teclado
  precio: 15000
```

Equivalente en JSON:

```
{
  "producto": {
    "codigo": 1,
    "descripcion": "Teclado",
    "precio": 15000
  }
}
```

Diccionario anidado:

```
usuario:
  nombre: Juan
  edad: 30
  direccion:
    calle: San Martín
    numero: 1234
    ciudad: Córdoba
```

Recuerda: cada nivel anidado se representa con dos espacios adicionales para mantener la jerarquía visual.

5.3 Combinación de listas y diccionarios

Las estructuras reales combinan listas y diccionarios para expresar relaciones más ricas.

Lista de diccionarios:

```
productos:
  - codigo: 1
    descripcion: Teclado
    precio: 15000
  - codigo: 2
    descripcion: Mouse
    precio: 9000
  - codigo: 3
```

```
    descripcion: Monitor
    precio: 85000
```

Diccionario con listas:

```
persona:
  nombre: Laura
  hobbies:
    - Leer
    - Correr
    - Programar
```

Lista de diccionarios anidados:

```
usuarios:
  - nombre: Ana
    edad: 25
    habilidades:
      - Python
      - Django
  - nombre: Marcos
    edad: 32
    habilidades:
      - Java
      - Spring Boot
```

5.4 Buenas prácticas

•Para listas, usa guiones con un espacio y alinea los elementos en el mismo nivel.

•En diccionarios, mantén las claves en minúsculas y consistentes (nombre, precio, codigo).

•En estructuras mixtas, respeta la indentación para que YAML pueda interpretar la jerarquía.

•Usa listas de diccionarios para colecciones de objetos similares (productos, usuarios).

•Utiliza diccionarios con listas cuando una clave agrupa múltiples valores (hobbies, habilidades).

Con listas, diccionarios y sus combinaciones puedes modelar desde datos simples hasta configuraciones complejas de forma clara y mantenible.

Referencias y anclas (& y *)

Una de las características más poderosas de YAML es la posibilidad de reutilizar bloques mediante anclas (&) y referencias o alias (*). Esto evita duplicación y mantiene la coherencia en configuraciones grandes.

6.1 Anclas (&)

Un ancla se define con el símbolo & seguido de un nombre y marca un bloque reutilizable.

```
base: &valores_base
  memoria: 4GB
  cpu: 2
  disco: 50GB
```

6.2 Referencias o alias (*)

Una referencia utiliza * con el nombre del ancla para copiar el bloque. Con el operador de merge (<<) se fusiona en el diccionario actual.

```
servidor1:
  <<: *valores_base
  nombre: servidor-app

servidor2:
  <<: *valores_base
  nombre: servidor-db
```

Resultado expandido:

```
servidor1:
  memoria: 4GB
  cpu: 2
  disco: 50GB
  nombre: servidor-app

servidor2:
  memoria: 4GB
  cpu: 2
  disco: 50GB
  nombre: servidor-db
```

6.3 Uso con listas

Los alias también funcionan sobre listas completas.

```
usuarios: &usuarios_base
  - nombre: Ana
```

```
    rol: admin
  - nombre: Marcos
    rol: editor

equipoA:
  miembros: *usuarios_base

equipoB:
  miembros: *usuarios_base
```

6.4 Sobrescritura de valores

Podemos partir de un ancla y sobrescribir campos específicos en cada alias.

```
config_base: &config_base
  puerto: 8080
  modo: desarrollo
  debug: true

config_prod:
  <<: *config_base
  modo: produccion
  debug: false
```

Resultado:

```
config_prod:
  puerto: 8080
  modo: produccion
  debug: false
```

6.5 Caso práctico: Docker Compose

En Docker Compose es habitual reutilizar bloques de configuración.

```
version: "3.9"

x-service-base: &service_base
  restart: always
  networks:
    - backend

services:
  app1:
    <<: *service_base
    image: miapp:1.0
    ports:
      - "8080:8080"
```

```yaml
  app2:
    <<: *service_base
    image: miapp:2.0
    ports:
      - "9090:8080"

networks:
  backend:
```

6.6 Buenas prácticas

•Nombre anclas con intención (&config_base, &valores_base) para claridad.
•Usa merge keys (<<: *alias) para modificar solo algunas claves.
•Evita un exceso de anclas y referencias para no dificultar la lectura.
•Resultan especialmente útiles en configuraciones de Kubernetes, Docker Compose y pipelines de CI/CD.

Con anclas y referencias, YAML permite reutilizar estructuras, reducir duplicación y mantener archivos más ordenados.

Patrones de reutilización con YAML

La reutilización es una de las mayores ventajas de YAML. Con anclas (&), alias (*) y el operador de merge (<<) evitamos duplicaciones, simplificamos configuraciones y mantenemos la coherencia en archivos extensos.

7.1 Reutilización de configuraciones base

Definimos una configuración común y la adaptamos para cada entorno.

```yaml
config_base: &config_base
  puerto: 8080
  host: localhost
  debug: true

config_test:
  <<: *config_base
  host: test.miapp.com

config_prod:
  <<: *config_base
  host: miapp.com
  debug: false
```

Beneficio: no repetimos claves como puerto o debug; solo sobrescribimos lo que cambia en cada entorno.

7.2 Reutilización en listas

Los alias también sirven para listas cuyos elementos comparten valores por defecto.

```yaml
usuario_base: &usuario_base
  rol: editor
  activo: true

usuarios:
  - nombre: Ana
    <<: *usuario_base
  - nombre: Marcos
    <<: *usuario_base
  - nombre: Laura
    <<: *usuario_base
    rol: admin
```

7.3 Reutilización de metadatos

En ecosistemas como Kubernetes es común compartir etiquetas y namespaces.

```yaml
metadata_base: &metadata_base
  labels:
    app: mi-app
  namespace: produccion

---
apiVersion: v1
kind: ConfigMap
metadata:
  <<: *metadata_base
  name: mi-config

---
apiVersion: v1
kind: Service
metadata:
  <<: *metadata_base
  name: mi-service
```

7.4 Reutilización en Docker Compose

En Docker Compose podemos centralizar políticas y redes.

```yaml
version: "3.9"

x-service-base: &service_base
  restart: always
```

```yaml
  networks:
    - backend

services:
  app1:
    <<: *service_base
    image: miapp:1.0
    ports:
      - "8080:8080"

  app2:
    <<: *service_base
    image: miapp:2.0
    ports:
      - "9090:8080"

networks:
  backend:
```

7.5 Patrón de valores por defecto

Definimos un bloque &defaults con parámetros comunes y lo extendemos según necesidad.

```yaml
defaults: &defaults
  timeout: 30
  retries: 3

servicio_a:
  <<: *defaults
  endpoint: /api/v1/usuarios

servicio_b:
  <<: *defaults
  endpoint: /api/v1/productos
  timeout: 60
```

7.6 Buenas prácticas al reutilizar

•Define bloques claros y con nombres expresivos (&config_base, &service_base).

•Usa merge keys (<<: *alias) para sobrescribir únicamente lo necesario.

•No abuses de las referencias; demasiadas pueden complicar la lectura.

•Añade comentarios que indiquen dónde se encuentran los bloques base.

7.7 Ventajas de estos patrones

•Evitan la duplicación de datos.

•Permiten modificar configuraciones globales en un único lugar.
•Facilitan la gestión de entornos (dev, test, prod).
•Son especialmente útiles en flujos DevOps como Docker Compose, Kubernetes y CI/CD.
En conclusión, aprovechar las anclas y alias para definir patrones de reutilización convierte a YAML en una herramienta más limpia, mantenible y escalable.

Documentos múltiples (---)

Un archivo de YAML puede contener varios documentos independientes separados por la línea especial ---. Cada bloque funciona como un documento distinto, lo que ayuda a agrupar configuraciones relacionadas sin dividirlas en múltiples archivos.

8.1 ¿Cuándo usar documentos múltiples?

•Agrupar configuraciones relacionadas que comparten contexto.
•Definir varios recursos de Kubernetes (Pods, Services, Deployments) en un solo archivo.
•Manejar configuraciones complejas donde conviene centralizar todo en un fichero.

8.2 Ejemplo simple

```
---
producto:
  codigo: 1
  descripcion: Teclado
  precio: 15000
---
producto:
  codigo: 2
  descripcion: Mouse
  precio: 9000
```

Cada bloque separado por --- representa un documento distinto. Algunos parsers leerán solo el primero, mientras que otros procesarán todos.

8.3 Ejemplo en Kubernetes

En Kubernetes es habitual definir varios recursos en una sola entrega.

```
---
apiVersion: v1
kind: ConfigMap
```

```yaml
metadata:
  name: mi-config
data:
  DB_HOST: localhost
  DB_PORT: "5432"
---
apiVersion: v1
kind: Service
metadata:
  name: mi-servicio
spec:
  selector:
    app: mi-app
  ports:
    - port: 80
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mi-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mi-app
  template:
    metadata:
      labels:
        app: mi-app
    spec:
      containers:
        - name: mi-app
          image: mi-app:1.0
          ports:
            - containerPort: 8080
```

De esta forma podemos aplicar todo con un solo comando: `kubectl apply -f recursos.yaml`.

8.4 Separador final (...)

Además de ---, YAML define ... para indicar explícitamente el final de un documento. Aunque es opcional, puede ser útil en flujos donde se encadenan múltiples documentos.

```
---
nombre: Juan
edad: 30
...
---
nombre: Laura
edad: 25
...
```

8.5 Buenas prácticas

•Usa --- para separar documentos relacionados manteniendo un orden lógico (ej.: ConfigMaps → Services → Deployments).

•Documenta con comentarios qué representa cada bloque para facilitar el mantenimiento.

•No mezcles configuraciones sin relación; dividir en varios archivos puede ser más claro.

•Comprueba que la herramienta que procesará el archivo soporte múltiples documentos.

8.6 Ventajas

•Agrupar múltiples configuraciones en un solo archivo.

•Facilitar despliegues en entornos DevOps con un único apply o carga.

•Reducir la cantidad de archivos sueltos y mejorar la organización.

8.7 Limitaciones

•No todos los parsers YAML admiten múltiples documentos.

•Un archivo demasiado extenso puede volverse difícil de leer si contiene muchos bloques.

En resumen, con --- podemos gestionar varios documentos en un mismo archivo YAML, una técnica clave en Kubernetes, Docker Compose avanzado y pipelines CI/CD donde conviene centralizar configuraciones.

Tags de tipo (!!str, !!int, etc.)

De forma predeterminada, YAML interpreta valores según su forma: 123 como entero, true como booleano o null/~ como nulo. Sin embargo, a veces necesitamos forzar un tipo específico sin dejar lugar a ambigüedades. Para eso existen los tags de tipo, que se escriben con el prefijo !!.

9.1 ¿Qué es un tag de tipo?

Un tag explicita el tipo que debe asignarse a un valor, independientemente de su forma textual.

```
valor1: !!str 123     # Se interpreta como cadena
valor2: !!int "123"   # Se interpreta como entero
```

9.2 Principales tags de tipo

a) !!str — cadenas de texto

```
codigo: !!str 00123
respuesta: !!str true
```

Útil para conservar ceros a la izquierda o tratar “true” como texto y no como booleano.

b) !!int — enteros

```
edad: !!int "30"
cantidad: !!int 100
```

Asegura que un número dentro de comillas se convierta en entero.

c) !!float — decimales

```
precio: !!float "199.99"
descuento: !!float 0.15
```

d) !!bool — booleanos

```
activo: !!bool "true"
deshabilitado: !!bool "false"
```

e) !!null — valores nulos

```
direccion: !!null ""
telefono: !!null
```

f) !!seq — listas (secuencias)

```
colores: !!seq [Rojo, Verde, Azul]
```

g) !!map — diccionarios (mapas)

```
producto: !!map
  codigo: 1
  descripcion: Monitor
  precio: 85000
```

9.3 Casos prácticos

•Códigos numéricos con ceros iniciales: codigo_postal: !!str 05000.
•Evitar ambigüedad con booleanos: respuesta: !!str yes para almacenarlo como texto.
•Validaciones estrictas: al integrar YAML con esquemas (JSON Schema, XML), los tags evitan errores de conversión.

9.4 Buenas prácticas

•Usa tags solo cuando sea necesario; YAML ya interpreta correctamente la mayoría de los casos.
•Prefiere !!str en valores ambiguos como 00123, yes, no.
•En proyectos grandes (Kubernetes, Docker, CI/CD), utilízalos solo si la herramienta de validación lo requiere.

9.5 Ventajas y limitaciones

Ventajas:

- Evitan errores de interpretación automática.
- Ofrecen mayor control sobre los datos.
- Resultan útiles en validaciones estrictas.

Limitaciones:

- No todos los parsers soportan cada tag.
- El uso excesivo puede reducir la legibilidad.

En resumen, los tags de tipo (!!str, !!int, !!float, !!bool, !!null, !!seq, !!map) nos permiten controlar con precisión cómo se interpretan los datos en YAML, evitando ambigüedades y garantizando que cada valor se trate con el tipo correcto.

Bloques literales y doblados

YAML permite representar texto multilínea sin recurrir a comillas. Los operadores | (bloque literal) y > (bloque doblado) facilitan la escritura de mensajes largos, documentación interna, logs o fragmentos de código.

Bloque literal (|) conserva los saltos de línea tal como están. Bloque doblado (>) convierte los saltos de línea simples en espacios para generar párrafos corridos.

10.1 Bloque literal (|)

El texto se mantiene exactamente como se escribe en el archivo, preservando cada salto de línea.

```
mensaje: |
  Estimado usuario:

  Su pedido ha sido recibido.
  En breve le enviaremos una confirmación por correo electrónico.

  Saludos cordiales,
  El equipo de Ventas
```

Resultado:

```
Estimado usuario:

Su pedido ha sido recibido.
En breve le enviaremos una confirmación por correo electrónico.

Saludos cordiales,
El equipo de Ventas
```

Ideal para mensajes formateados, logs, documentación interna o bloques de código.

10.2 Bloque doblado (>)

Con > los saltos de línea simples se reemplazan por espacios; se conservan únicamente los saltos dobles.

```
descripcion: >
  Este es un teclado mecánico.
  Tiene retroiluminación RGB.
  Ideal para gaming.
```

Resultado: Este es un teclado mecánico. Tiene retroiluminación RGB. Ideal para gaming.

Útil para descripciones largas que deben almacenarse en una sola línea.

10.3 Comparación rápida

```
texto_literal: |
  Línea 1
  Línea 2
  Línea 3
```

```
texto_doblado: >
  Línea 1
  Línea 2
  Línea 3
```

Literal (|): muestra

```
Línea 1
Línea 2
Línea 3
```

Doblado (>): muestra Línea 1 Línea 2 Línea 3

10.4 Control de saltos finales (|+, |-, >+, >-)

Los modificadores permiten decidir si se conservan o eliminan los saltos de línea al final del bloque.

```
texto1: |-
  Hola
  Mundo

texto2: |+
  Hola
  Mundo
```

- |- elimina el salto final.
- |+ conserva el salto final.
- Los mismos modificadores se aplican a >.

10.5 Buenas prácticas

- Usa bloques literales (|) cuando los saltos de línea sean relevantes (logs, código, correos).
- Usa bloques doblados (>) para párrafos largos que deben almacenarse en una sola línea.
- Combina + o - para controlar el comportamiento de las líneas finales.
- Mantén una indentación consistente dentro del bloque para evitar errores.

10.6 Casos de uso

Logs o mensajes de error (con |):

```
error_log: |
  Error 500: Internal Server Error
  Traceback (most recent call last):
    File "app.py", line 10, in <module>
```

Descripciones largas en DevOps (con >):

```
descripcion: >
```

```
  Este pipeline ejecuta las pruebas unitarias,
  construye la imagen Docker y
  despliega la aplicación en Kubernetes.
```

En resumen, | preserva los saltos de línea y > los convierte en espacios. Los modificadores + y - nos dan control adicional sobre las líneas finales, permitiendo ajustar el formato según la necesidad.

Archivos de configuración de aplicaciones

Uno de los usos más extendidos de YAML es la configuración de aplicaciones y servicios. Su sintaxis clara lo volvió el formato predilecto en herramientas de DevOps como Docker Compose y Kubernetes.

11.1 Docker Compose (docker-compose.yml)

¿Qué es Docker Compose? Permite definir y ejecutar aplicaciones multi-contenedor desde un único archivo YAML que describe servicios, redes y volúmenes.

Ejemplo: aplicación web con base de datos

```yaml
version: "3.9"

services:
  app:
    image: miapp:1.0
    container_name: app_container
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASSWORD=1234
    depends_on:
      - db

  db:
    image: mysql:8.0
    container_name: db_container
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: miappdb
    volumes:
      - db_data:/var/lib/mysql

volumes:
```

```
  db_data:
```

Explicación:

•version: versión del esquema.

•services: define los contenedores (app y db).

•app: servicio que expone el puerto 8080.

•db: servicio MySQL con volumen persistente.

•volumes: asegura que los datos sobrevivan a reinicios.

Ejecución: docker-compose up -d levanta todo el stack.

11.2 Kubernetes (deployment.yaml y service.yaml)

Kubernetes describe el estado deseado de pods, servicios, deployments, configmaps y más a través de archivos YAML.

Deployment (deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mi-app
  labels:
    app: mi-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mi-app
  template:
    metadata:
      labels:
        app: mi-app
    spec:
      containers:
        - name: mi-app-container
          image: miapp:1.0
          ports:
            - containerPort: 8080
```

Claves: replicas define la cantidad de instancias; selector y labels enlazan el Deployment con los pods correctos; containers especifica imagen y puertos.

Aplicación: kubectl apply -f deployment.yaml

Service (service.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mi-app-service
spec:
  selector:
    app: mi-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
```

Claves: selector conecta el Service con los pods etiquetados app: mi-app; ports define la redirección; type determina el alcance (interno con ClusterIP).

Aplicación: kubectl apply -f service.yaml

11.3 Deployment y Service juntos

Podemos declarar varios objetos en un único archivo usando separadores ---.

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mi-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mi-app
  template:
    metadata:
      labels:
        app: mi-app
    spec:
      containers:
        - name: mi-app-container
          image: miapp:1.0
          ports:
            - containerPort: 8080
---
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mi-app-service
spec:
  selector:
    app: mi-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
```

Aplicación conjunta: kubectl apply -f mi-app.yaml

11.4 Docker Compose vs Kubernetes

Aspecto	Docker Compose	Kubernetes
Enfoque	Despliegue local y sencillo	Orquestación de clústeres a gran escala
Archivo YAML	`docker-compose.yml`	Archivos por recurso (`deployment.yaml`, `service.yaml`, etc.)
Escalabilidad	Limitada (escala manual)	Escalado automático con réplicas/autoscaling
Complejidad	Fácil de aprender	Más complejo y potente

11.5 Buenas prácticas

•Mantén la indentación uniforme para evitar errores de despliegue.

•Documenta los bloques con comentarios claros.

•Divide configuraciones grandes en varios archivos (por ejemplo: deployment.yaml, service.yaml, configmap.yaml).

•Usa variables de entorno para ocultar credenciales.

•Versiona los YAML en Git para auditoría y CI/CD.

En definitiva, YAML es la base de la configuración en Docker Compose y Kubernetes, pilares del ecosistema DevOps. Dominar archivos como docker-compose.yml, deployment.yaml y service.yaml es esencial para desplegar aplicaciones modernas.

Uso en frameworks y lenguajes

Además del mundo DevOps, YAML es ampliamente utilizado como formato de configuración o fuente de datos en distintos lenguajes y frameworks. A continuación,

vemos tres casos comunes: Spring Boot (Java), Python con PyYAML y Node.js con js-yaml.

12.1 Spring Boot

Spring Boot soporta application.properties o application.yml. El formato YAML resulta más legible gracias a su estructura jerárquica.

Ejemplo application.yml:

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/miappdb
    username: root
    password: 1234
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

logging:
  level:
    org.springframework: INFO
    com.miapp: DEBUG
```

Claves: server.port configura el puerto; spring.datasource define la conexión a la base; spring.jpa controla Hibernate; logging.level ajusta niveles de log por paquete. Es habitual crear perfiles como application-dev.yml o application-prod.yml.

12.2 Python con PyYAML

Instalación: pip install pyyaml

Archivo config.yml:

```yaml
app:
  nombre: Mi Aplicación
  version: 1.0
  debug: true

database:
  host: localhost
  port: 5432
  user: admin
  password: secreto
```

Lectura en Python:

```python
import yaml

with open("config.yml", "r", encoding="utf-8") as f:
    config = yaml.safe_load(f)

print("Nombre de la app:", config["app"]["nombre"])
print("Modo debug:", config["app"]["debug"])
print("Base de datos:", config["database"]["host"], config["database"]["port"])
```

`yaml.safe_load()` convierte el YAML en un diccionario. Para escribir YAML:

```python
data = {
    "usuarios": [
        {"nombre": "Ana", "rol": "admin"},
        {"nombre": "Marcos", "rol": "editor"}
    ]
}

with open("usuarios.yml", "w", encoding="utf-8") as f:
    yaml.dump(data, f, allow_unicode=True)
```

12.3 Node.js con js-yaml

Instalación: `npm install js-yaml`

Archivo `config.yml`:

```yaml
server:
  port: 3000

auth:
  enabled: true
  secret: "clave-secreta"
```

Lectura en Node.js:

```node.js
const fs = require("fs");
const yaml = require("js-yaml");

try {
  const fileContents = fs.readFileSync("config.yml", "utf8");
  const config = yaml.load(fileContents);

  console.log("Servidor en puerto:", config.server.port);
  console.log("Auth habilitado:", config.auth.enabled);
} catch (e) {
  console.error("Error al leer YAML:", e);
}
```

Para generar YAML:

```
const usuarios = [
  { nombre: "Ana", rol: "admin" },
  { nombre: "Marcos", rol: "editor" }
];

const yamlString = yaml.dump({ usuarios });
fs.writeFileSync("usuarios.yml", yamlString, "utf8");
```

12.4 Comparación rápida

Lenguaje / Framework	Librería / Soporte	Uso principal
Spring Boot (Java)	Soporte nativo con `application.yml`	Configuración empresarial
Python	PyYAML (`pip install pyyaml`)	Lectura/escritura de configuraciones
Node.js	js-yaml (`npm install js-yaml`)	Configuraciones en apps web o CLI

12.5 Buenas prácticas

•Versiona los archivos YAML junto con el código del proyecto.
•Evita credenciales en texto plano; usa variables de entorno o gestores de secretos.
•Valida el YAML antes de usarlo para evitar errores de indentación.
•Separa configuraciones por entornos (dev, test, prod).

En síntesis, YAML no es solo clave en DevOps: frameworks como Spring Boot y librerías como PyYAML y js-yaml permiten manejar configuraciones limpias y portables en distintas plataformas.

Validación y linting de YAML

YAML es sensible a la indentación y los espacios. Un solo error puede invalidar un despliegue en Docker Compose o Kubernetes. Las herramientas de validación y linting ayudan a detectar fallos, aplicar reglas de estilo y garantizar consistencia antes de ejecutar sistemas críticos.

13.1 Herramientas automáticas

a) YAML Lint (yamllint CLI)

Instalación: pip install yamllint
Validación:
yamllint config.yml

Muestra la línea y columna de errores, además de sugerencias de estilo. Existe versión online en YAML Lint.

b) Extensión YAML de Red Hat (VSCode)

Disponible en el Marketplace de VSCode, brinda validación en tiempo real, autocompletado basado en esquemas y sugerencias de corrección. Ideal para trabajar con Kubernetes, GitHub Actions o Docker.

c) Validación con JSON Schema

Muchos parsers permiten validar YAML contra esquemas JSON. Ejemplo: validar deployment.yaml contra el esquema oficial de apps/v1, garantizando estructura y tipos correctos.

d) Validación en pipelines CI/CD

Integrar validadores en automatizaciones evita que YAML erróneos lleguen a producción.

```yaml
name: Validar YAML

on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Instalar yamllint
        run: pip install yamllint
      - name: Validar archivos YAML
        run: yamllint .
```

13.2 Errores comunes

Uso de tabulaciones

Incorrecto:

```yaml
producto:
codigo: 1  # tabulación
```

Correcto:

```yaml
producto:
  codigo: 1
```

Indentación inconsistente

Incorrecto:

```
usuario:
  nombre: Ana
   edad: 25
```

Correcto:

```
usuario:
  nombre: Ana
  edad: 25
```

Valores ambiguos sin comillas

Recomendado:

```
respuesta: "yes"
codigo_postal: "05000"
```

Separadores de documentos mal usados

Incorrecto:

```
---
producto: Teclado
---producto: Mouse
```

Correcto:

```
---
producto: Teclado
---
producto: Mouse
```

Listas mal indentadas

Incorrecto:

```
frutas:
- Manzana
  - Banana
- Naranja
```

Correcto:

```
frutas:
  - Manzana
  - Banana
  - Naranja
```

13.3 Buenas prácticas

•Utiliza 2 espacios por nivel de indentación (nunca tabs).

•Valida los archivos con yamllint o la extensión de VSCode antes de desplegar.
•Usa comillas en valores ambiguos ("yes", "no", "00123").
•Mantén un estilo consistente (claves en minúsculas, estructura clara).
•Versiona los YAML y agrega validación automática en los pipelines.
En conclusión, la validación y el linting son fundamentales para asegurar que los archivos YAML sean correctos, consistentes y estén listos para sistemas sensibles como Kubernetes, Docker Compose o workflows de CI/CD.

Conversión entre YAML y JSON

YAML y JSON son formatos populares para datos estructurados. YAML es un superconjunto de JSON, por lo que cualquier JSON válido también es YAML válido. Esto facilita la conversión entre ambos según la necesidad: hacer el archivo más legible (JSON → YAML) o integrarlo con APIs (YAML → JSON).

14.1 Ejemplo de equivalencia

JSON:

```
{
  "productos": [
    { "codigo": 1, "descripcion": "Teclado", "precio": 15000 },
    { "codigo": 2, "descripcion": "Mouse", "precio": 9000 }
  ]
}
```

YAML equivalente:

```
productos:
  - codigo: 1
    descripcion: Teclado
    precio: 15000
  - codigo: 2
    descripcion: Mouse
    precio: 9000
```

Ambos representan una lista de objetos con las mismas claves.

14.2 Ventajas al convertir

JSON → YAML:

- Más legible para humanos.
- Menos caracteres ({}, [], comas).
- Óptimo para configuraciones.

YAML → JSON:

- Ideal para APIs REST y microservicios.

- Compatible con la mayoría de librerías.
- Formato más estricto (reduce errores de indentación).

14.3 Herramientas online

•JSON ↔️ YAML Converter

•Online YAML Tools

•ConvertJSON

Permiten pegar texto y obtener la conversión sin instalar nada.

14.4 Conversión en Python (PyYAML)

JSON → YAML:

```
import yaml
import json

json_data = '{"nombre": "Ana", "edad": 25, "hobbies": ["leer", "correr"]}'
data = json.loads(json_data)
yaml_data = yaml.dump(data, allow_unicode=True)

print(yaml_data)
```

Resultado:

```
edad: 25
hobbies:
- leer
- correr
nombre: Ana
```

YAML → JSON:

```
yaml_data = """
nombre: Ana
edad: 25
hobbies:
  - leer
  - correr
"""

data = yaml.safe_load(yaml_data)
json_data = json.dumps(data, indent=2, ensure_ascii=False)

print(json_data)
```

14.5 Conversión en Node.js (js-yaml)

JSON → YAML:

```
const yaml = require("js-yaml");
```

```
const jsonData = {
  nombre: "Ana",
  edad: 25,
  hobbies: ["leer", "correr"]
};

const yamlStr = yaml.dump(jsonData);
console.log(yamlStr);
```

YAML → JSON:

```
const fs = require("fs");
const yaml = require("js-yaml");

const yamlStr = `
nombre: Ana
edad: 25
hobbies:
  - leer
  - correr
`;

const data = yaml.load(yamlStr);
console.log(JSON.stringify(data, null, 2));
```

14.6 Buenas prácticas

•Usa YAML para configuraciones legibles y JSON para intercambio de datos en APIs.
•Convierte con librerías oficiales (PyYAML, js-yaml) para evitar errores manuales.
•Valida el archivo después de convertirlo.
•Recuerda que YAML soporta comentarios (#), pero JSON no.

En resumen, convertir entre YAML y JSON es sencillo gracias a su relación directa. Aprovecha YAML para configuraciones comprensibles y JSON para comunicación entre sistemas, eligiendo el formato más adecuado en cada contexto.

Estilo y convenciones de nombres en proyectos reales

YAML es flexible, pero esa flexibilidad puede volverse un problema si cada persona escribe con su propio estilo. En proyectos grandes (microservicios, DevOps, CI/CD, Kubernetes) conviene establecer convenciones claras para mantener orden y evitar errores.

15.1 Convenciones para las claves

•Usa siempre minúsculas.

•Separa palabras con guion medio (-) o guion bajo (_).

•Evita nombres largos o ambiguos.

```
app_name: mi-aplicacion
db_host: localhost
max_connections: 50
```

Evita estilos mixtos como AppName o DBHOST que dificultan la lectura.

15.2 Indentación consistente

•Solo espacios, nunca tabs.

•Convención más común: 2 espacios por nivel.

```
server:
  port: 8080
  host: localhost
```

Asegúrate de no mezclar 2 y 4 espacios en el mismo bloque.

15.3 Listas alineadas

Cada elemento debe comenzar con - seguido de un espacio.

```
frutas:
  - manzana
  - banana
  - naranja
```

15.4 Uso de comillas

No son obligatorias, pero se recomiendan cuando:

•Hay caracteres especiales (:, #, {}).

•El valor puede interpretarse como booleano (yes, no, on, off).

•El valor comienza con ceros.

```
codigo_postal: "05000"
respuesta: "yes"
descripcion: "Producto: Teclado # edición limitada"
```

15.5 Orden lógico de las claves

En archivos extensos conviene definir un orden por secciones (configuración general, bases de datos, servicios externos, logs).

```
app:
  name: mi-aplicacion
  version: 1.0
```

```yaml
database:
  host: localhost
  port: 5432
  user: admin
  password: secreto

logging:
  level:
    root: INFO
    com.miapp: DEBUG
```

15.6 Convenciones en proyectos DevOps

Docker Compose: nombres de servicios en minúsculas sin espacios.

```yaml
services:
  web:
    image: nginx
  db:
    image: mysql
```

Kubernetes: metadata.name en minúsculas con guiones, labels y selectors consistentes.

```yaml
metadata:
  name: mi-app-service
  labels:
    app: mi-app
```

15.7 Comentarios útiles

•Explica configuraciones clave.

•Evita comentarios triviales.

•Coloca comentarios antes del bloque, no al final.

```yaml
# Configuración de la base de datos
database:
  host: localhost
  port: 5432
```

15.8 Integración con linters

Para asegurar consistencia:

•yamllint: valida sintaxis y estilo.

•EditorConfig: define reglas de indentación.

•Extensiones para VSCode como la de Red Hat.

15.9 Beneficios

- Archivos más legibles y mantenibles.
- Menos errores de interpretación.
- Facilita la colaboración en equipo.
- Alineado con buenas prácticas DevOps.

En síntesis, definir y seguir convenciones de estilo en YAML es clave para proyectos reales. Claves en minúsculas, listas alineadas, valores ambiguos entre comillas y una estructura ordenada minimizan errores y agilizan el trabajo en equipo.

Seguridad y limitaciones de YAML

YAML es omnipresente en configuraciones y DevOps, pero su flexibilidad trae riesgos. Un uso descuidado puede introducir vulnerabilidades, errores en producción o archivos difíciles de mantener.

16.1 Riesgos de seguridad

a) Ejecución de código malicioso

Algunas librerías (por ejemplo yaml.load en Python) pueden ejecutar código arbitrario.

```
!!python/object/apply:os.system ["rm -rf /"]
```

Si se procesa con yaml.load, el comando se ejecutará en el sistema.
Mitigación: usa safe_load() y evita procesar YAML no confiable sin validación.

b) Inyección de datos

Un atacante puede alterar configuraciones para habilitar debug o escalar privilegios.

```
debug: true
admin: true
```

Mitigación: valida valores críticos contra esquemas (JSON Schema, OpenAPI) e integra linters en CI/CD.

c) Exposición de credenciales

Incorrecto:

```
database:
  user: root
  password: 1234
```

Mitigación: usa variables de entorno o gestores de secretos (Vault, Kubernetes Secrets).

16.2 Limitaciones técnicas

a) Sensibilidad a la indentación

```
app:
    name: MiApp
  port: 8080
```

Un espacio de más puede invalidar todo el archivo.

b) Ambigüedad en valores

```
respuesta: yes  # Interpretado como true
```

Usa comillas para valores dudosos: `respuesta: "yes"`.

c) Archivos grandes y confusos

Los YAML extensos con múltiples anclas y alias son difíciles de revisar, especialmente en Kubernetes.

d) Comentarios limitados

Solo se admite `#` en una línea; no hay comentarios multilínea.

e) Rendimiento

Parsear YAML suele ser más lento que JSON, por lo que no es ideal para intercambio de datos en tiempo real.

16.3 Buenas prácticas de seguridad

•Usa lectores seguros (`safe_load` y equivalentes).
•Valida contra esquemas para garantizar tipos y valores.
•No almacenes credenciales en texto plano.
•Versiona los YAML, pero excluye datos sensibles.
•Integra validadores/linting en CI/CD.

16.4 Buenas prácticas ante limitaciones

•Divide archivos grandes en módulos manejables.
•Documenta cada sección con comentarios claros.
•Evita ambigüedades con comillas y nombres consistentes.
•Aplica el estilo uniforme definido (ver Tema 15).
•Usa JSON cuando el rendimiento sea crítico.

En resumen, YAML es potente pero requiere disciplina. Riesgos como ejecución de código, exposición de credenciales o errores de indentación pueden mitigarse con lectores seguros, validación, linters y convenciones claras.